TrustDeploy — Implementation Progress Log

This document records every significant step of the TrustDeploy implementation, with a screenshot taken immediately after each step. All screenshots come from a single clean run of scripts/run_all.sh on 2026-10-20 (local macOS, no cloud resources). Each one shows the exact command executed, its real output, its exit code and the capture time.

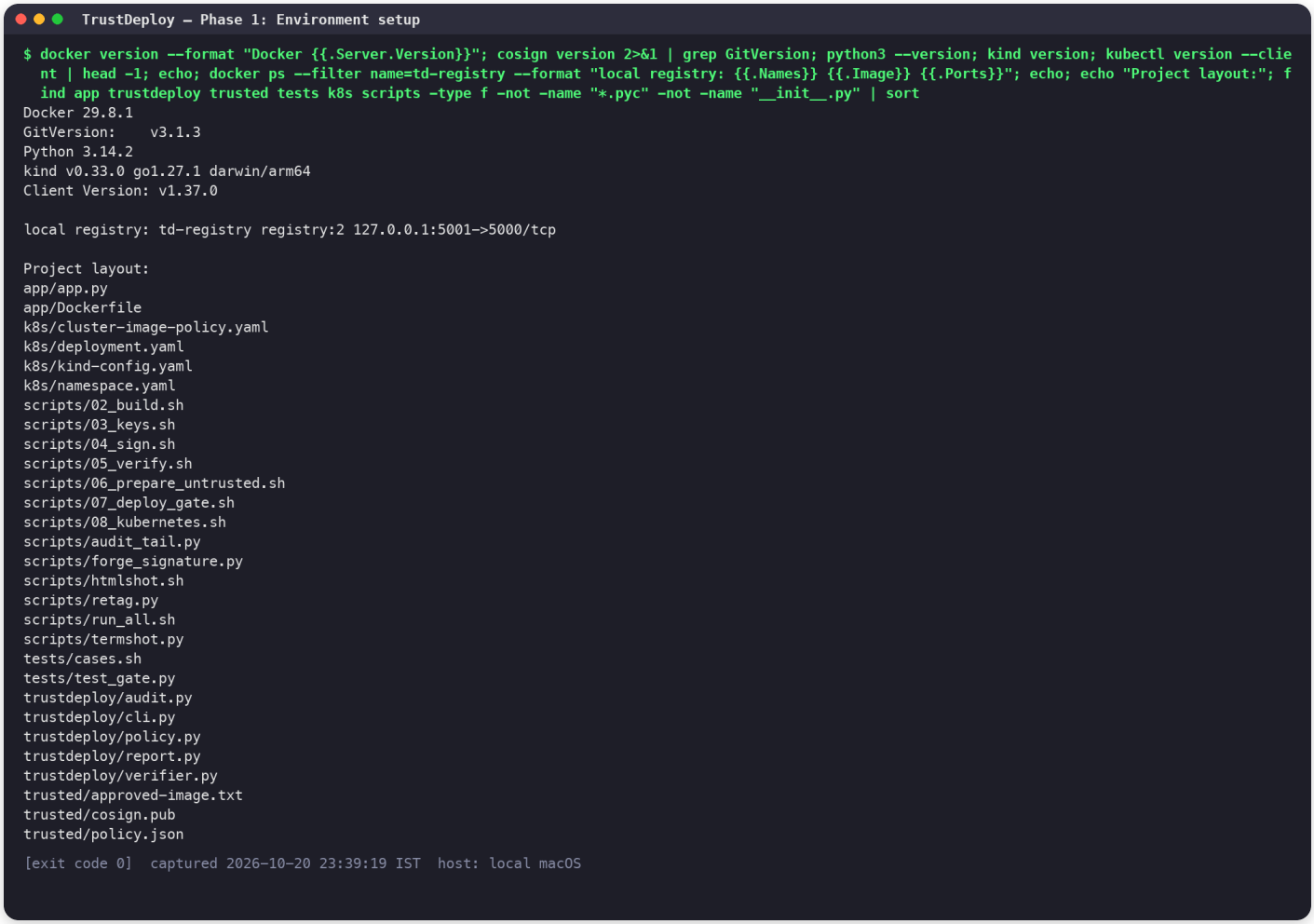
Test results summary

Test	Condition	Expected	Observed	Result
TC01	Correctly signed image	Accepted	ACCEPT — SIGNATURE_VERIFIED	**PASS**
TC02	Unsigned image	Rejected	REJECT — NO_SIGNATURE	**PASS**
TC03	Signed with untrusted key	Rejected	REJECT — UNTRUSTED_SIGNER	**PASS**
TC04	Signature does not match image (copied signature / tag hijack)	Rejected	REJECT — SIGNATURE_IMAGE_MISMATCH / NO_SIGNATURE	**PASS**
TC05	Tool error / registry down / registry not allowed	Deployment denied	REJECT — VERIFICATION_TOOL_ERROR / IMAGE_RESOLUTION_FAILED / REGISTRY_NOT_ALLOWED	**PASS**
TC06	Multiple images checked	Each recorded separately	4 images → 4 audit entries	**PASS**
TC07	Audit report generated	Displayed / exported	HTML + CSV + JSON written	**PASS**

Step-by-step log

1. Phase 1 — Environment setup

Docker, Cosign v3.1.3, Python, kind and kubectl are installed. A local OCI registry (registry:2) runs on localhost:5001, so no cloud resources are needed. The project tree separates the verifier, policy, audit and report components.



2. Phase 2 — Build the sample image and record its digest

The sample HTTP service is built and pushed with the versioned tag 1.0.0. The immutable manifest digest is recorded in trusted/approved-image.txt, and a smoke test confirms the container responds.

TrustDeploy — Phase 2: Build image & record digest

```
$ ./scripts/02_build.sh
Built and pushed: localhost:5001/trustdeploy/sample-app:1.0.0
Immutable digest: sha256:deffd203e38fc26a1c0fdb4a8d60a2de55884f4dc1946fc184d50bbe4a4c28b1

REPOSITORY                              TAG              IMAGE ID       SIZE
localhost:5001/trustdeploy/sample-app   1.0.0-hijack     8c2dddd74e9a   87.9MB
localhost:5001/trustdeploy/sample-app   1.0.2-forged     15fec4ab6a77   87.9MB
localhost:5001/trustdeploy/sample-app   1.0.1-attacker   d5b8709acdb3   87.9MB
localhost:5001/trustdeploy/sample-app   1.1.0-unsigned   b9e40137b00c   87.9MB
localhost:5001/trustdeploy/sample-app   1.0.0            deffd203e38f   87.9MB

Smoke test of the image:
{"service": "trustdeploy-sample-app", "version": "1.0.0", "status": "ok"}
[exit code 0]  captured 2026-10-20 23:39:23 IST  host: local macOS
```

3. Phase 3a — Generate the trusted signing key pair

Cosign generates an encrypted ECDSA P-256 key pair in ~/.trustdeploy/keys (mode 600, outside the project). Only the public key is copied into trusted/cosign.pub. Checks confirm there is no *.key file in the project tree or inside the image.

```
TrustDeploy — Phase 3: Generate trusted signing key

$ ./scripts/03_keys.sh
Private key (encrypted, never in repo/image):
  total 24
  -rw-------@ 1 amritnigam  staff  653 20 Oct 23:30 cosign.key
  -rw-r--r--@ 1 amritnigam  staff  178 20 Oct 23:30 cosign.pub
  -rw-------@ 1 amritnigam  staff   33 20 Oct 23:30 password

Trusted public key committed to the project: trusted/cosign.pub
-----BEGIN PUBLIC KEY-----
MFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAEY2b6FmBx0DgxS6U62bBEmCVBe5CK
AV4VHFUgHPELCYOUwm5WMSa4Y9tspN9Mj7+8kKT+Adohn6AeA8R/AHH3BA==
-----END PUBLIC KEY-----
SHA-256 fingerprint: e40f4da2c7a6c386

Is the private key anywhere in the project or the image?
  *.key files in project tree: 0
  *.key files inside image: 0

[exit code 0]  captured 2026-10-20 23:39:24 IST  host: local macOS
```

4. Phase 3b — Sign the approved image by digest

The approved image is signed by digest, never by tag. Two signatures are stored: the Sigstore bundle (OCI referrer) used by the TrustDeploy gate, and the legacy .sig format read by the Kubernetes Policy Controller. The transparency-log upload is disabled for this offline prototype.

```
TrustDeploy — Phase 3: Sign approved image with Cosign

$ ./scripts/04_sign.sh
Approved image (digest-pinned): localhost:5001/trustdeploy/sample-app@sha256:deffd203e38fc26a1c0fdb4a8d60a2de55884f4dc1946fc184d50bbe4a4c28b1

-- Sigstore bundle format (OCI referrer), used by the TrustDeploy gate
Signing artifact...
Pushing signature to: localhost:5001/trustdeploy/sample-app
-- legacy .sig format, read by the Kubernetes Policy Controller admission webhook
Signing artifact...
Pushing signature to: localhost:5001/trustdeploy/sample-app

Signature artifacts now attached to the image in the registry:
 Supply Chain Security Related artifacts for an image: localhost:5001/trustdeploy/sample-app@sha256:deffd203e38fc26a1c0fdb4a8d60a2de55884f4dc1946fc184
  d50bbe4a4c28b1
└── Signatures for an image tag: localhost:5001/trustdeploy/sample-app:sha256-deffd203e38fc26a1c0fdb4a8d60a2de55884f4dc1946fc184d50bbe4a4c28b1.sig
    └── sha256:a25e7b3e5194fe2c1025f65ae735a13da573a028d07b95105a0cc0c7645a98ae
└── https://sigstore.dev/cosign/sign/v1 artifacts via OCI referrer: localhost:5001/trustdeploy/sample-app@sha256:aca95f82041925223ee91ce893126adf1b72
  c9544f5b48a096b2517f070ea2f9
    └── sha256:a06dbb22d0d96ac2f86da7430a70bb40bc8ced1f0dcbd5ded6b5a18eeb0d8e68

[exit code 0]  captured 2026-10-20 23:39:26 IST  host: local macOS
```

5. Phase 4 — Verify the trusted image (TC01)

cosign verify validates the signature against trusted/cosign.pub. The TrustDeploy gate receives the mutable tag :1.0.0, resolves it to the digest, verifies it, checks that the signed payload names that exact digest, and returns ACCEPT. Result: TC01 passed.

```
TrustDeploy — Phase 4: Verify trusted image (TC01)
$ ./scripts/05_verify.sh
== cosign verify against trusted/cosign.pub ==

Verification for localhost:5001/trustdeploy/sample-app@sha256:deffd203e38fc26a1c0fdb4a8d60a2de55884f4dc1946fc184d50bbe4a4c28b1 --
The following checks were performed on each of these signatures:
  - The cosign claims were validated
  - Existence of the claims in the transparency log was verified offline
  - The signatures were verified against the specified public key

[{"critical":{"identity":{"docker-reference":"localhost:5001/trustdeploy/sample-app@sha256:deffd203e38fc26a1c0fdb4a8d60a2de55884f4dc1946fc184d50bbe4a4
  c28b1"},"image":{"docker-manifest-digest":"sha256:

== TrustDeploy gate, given the mutable tag :1.0.0 (resolved to digest first) ==
TrustDeploy policy gate  (policy: trustdeploy-default)
  image     : localhost:5001/trustdeploy/sample-app:1.0.0
  digest    : sha256:deffd203e38fc26a1c0fdb4a8d60a2de55884f4dc1946fc184d50bbe4a4c28b1
  decision  : ACCEPT  [SIGNATURE_VERIFIED]
  reason    : Valid signature from trusted key trusted/cosign.pub for sha256:deffd203e38f…
  audit id  : ebf0c89c-c68d-4bae-9255-50d3c4480a32  @ 2026-10-20T18:09:27+00:00
[exit code 0]  captured 2026-10-20 23:39:27 IST  host: local macOS
```

6. Phase 5 — Prepare untrusted / attack images

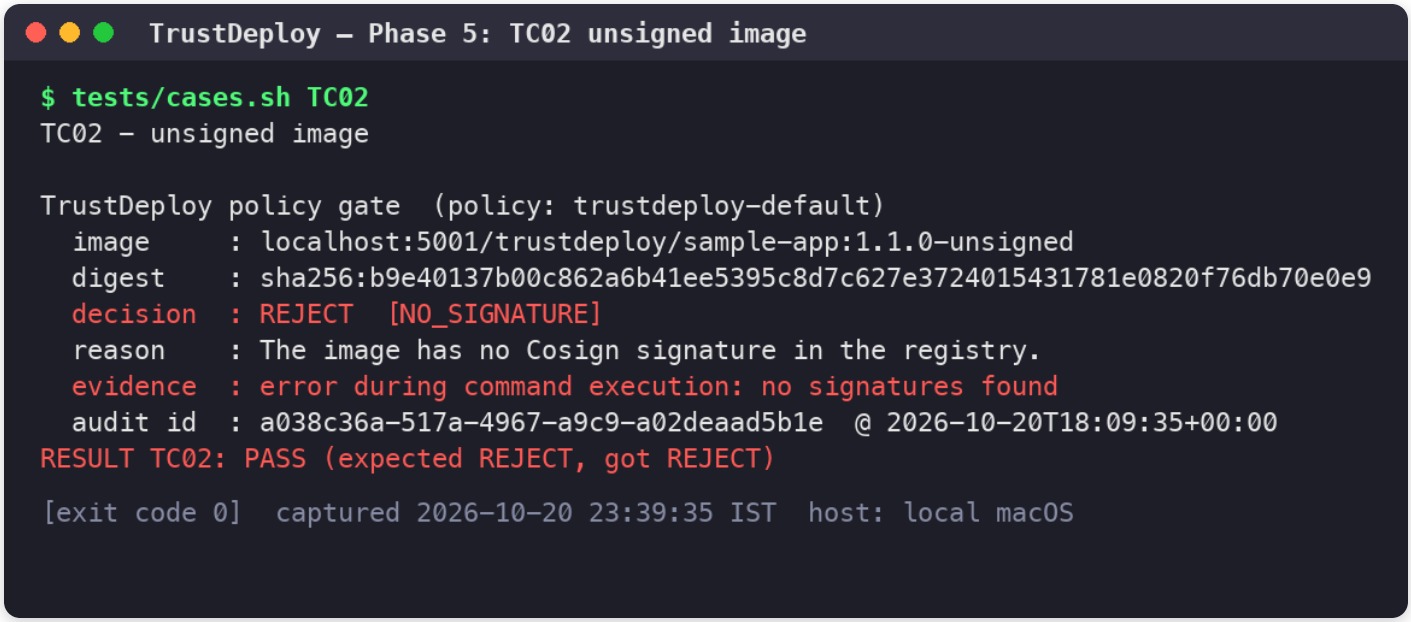
The negative tests use four images: an unsigned image; a backdoored image signed with an attacker's own key (a different fingerprint); an image carrying a genuine signature copied from 1.0.0 (scripts/forge_signature.py); and a tampered build used to hijack the :1.0.0 tag.

```
TrustDeploy — Phase 5: Prepare untrusted / attack images
$ ./scripts/06_prepare_untrusted.sh
[TC02] unsigned image  -> localhost:5001/trustdeploy/sample-app:1.1.0-unsigned
[TC03] attacker builds a malicious image and signs it with THEIR OWN key -> localhost:5001/trustdeploy/sample-app:1.0.1-attacker
Pushing signature to: localhost:5001/trustdeploy/sample-app
Pushing signature to: localhost:5001/trustdeploy/sample-app
   attacker key fingerprint: 98ef4a7fc9541039  (trusted: e40f4da2c7a6c386)
[TC04a] attacker copies the GENUINE signature of 1.0.0 onto their image -> localhost:5001/trustdeploy/sample-app:1.0.2-forged
Copied signature bundle layer sha256:a06dbb22d0d9… from sha256:deffd203e38f… onto sha256:15fec4ab6a77…
[TC04b] attacker builds a tampered 1.0.0 (pushed during the test over the trusted :1.0.0 tag)

Registry tags now present:
   1.0.0
   1.1.0-unsigned
   sha256-deffd203e38fc26a1c0fdb4a8d60a2de55884f4dc1946fc184d50bbe4a4c28b1
   sha256-d5b8709acdb347ffe5db0d485a93d6cc96bedec5ae0049dba426bb70d8f31678.sig
   1.0.2-forged
   sha256-15fec4ab6a7720deff992d74b5b06dd132fecaa3847144a7a5a5562023db1fbe
   sha256-deffd203e38fc26a1c0fdb4a8d60a2de55884f4dc1946fc184d50bbe4a4c28b1.sig
   1.0.1-attacker
   tampered-build
   sha256-d5b8709acdb347ffe5db0d485a93d6cc96bedec5ae0049dba426bb70d8f31678
[exit code 0]  captured 2026-10-20 23:39:35 IST  host: local macOS
```

7. TC02 — Unsigned image

Rejected with NO_SIGNATURE. Result: TC02 passed.



8. TC03 — Image signed with an untrusted key

A signature exists, but it does not validate against the trusted key, so the image is rejected with UNTRUSTED_SIGNER. Result: TC03 passed.



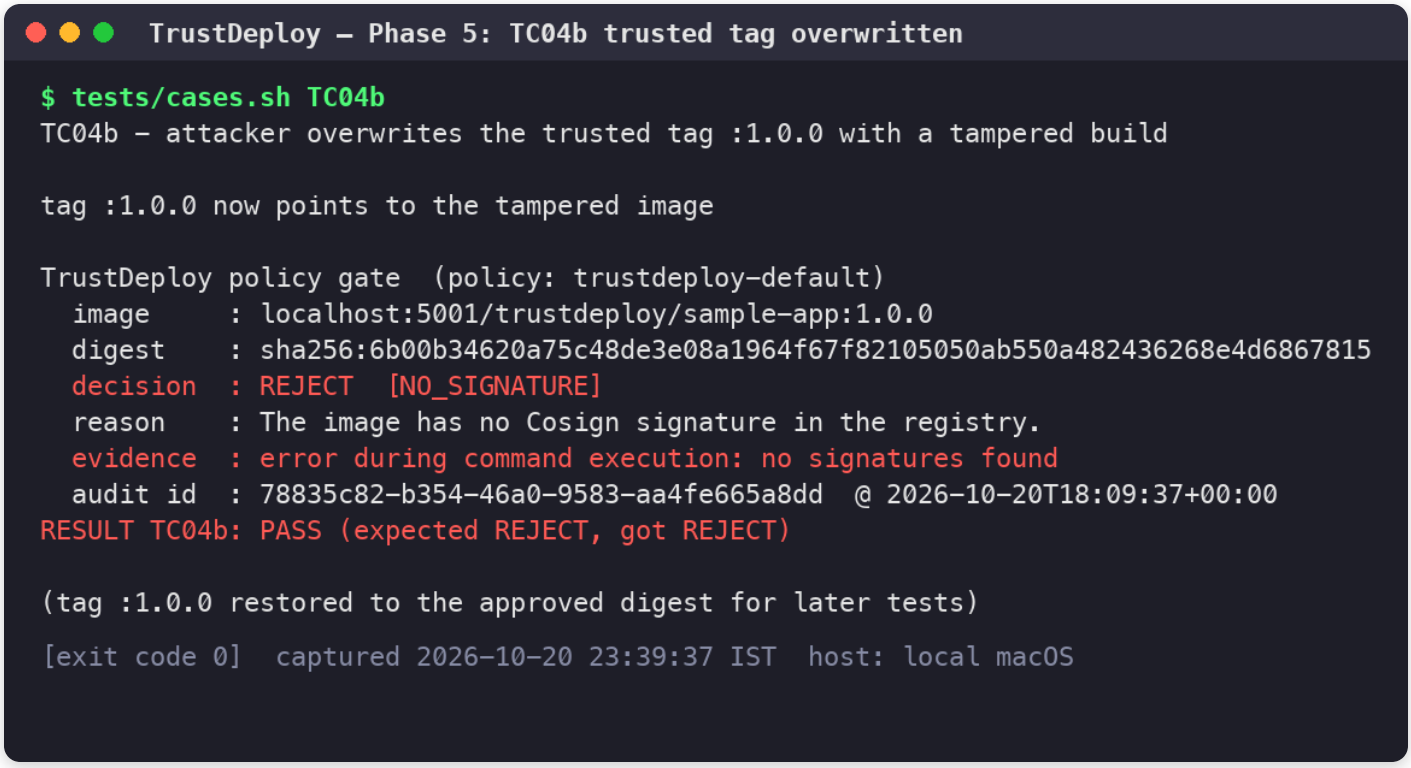
9. TC04a — Genuine signature attached to a different image

The copied signature is cryptographically valid, but its signed payload names a different digest, so the image is rejected with SIGNATURE_IMAGE_MISMATCH. Result: TC04 passed.



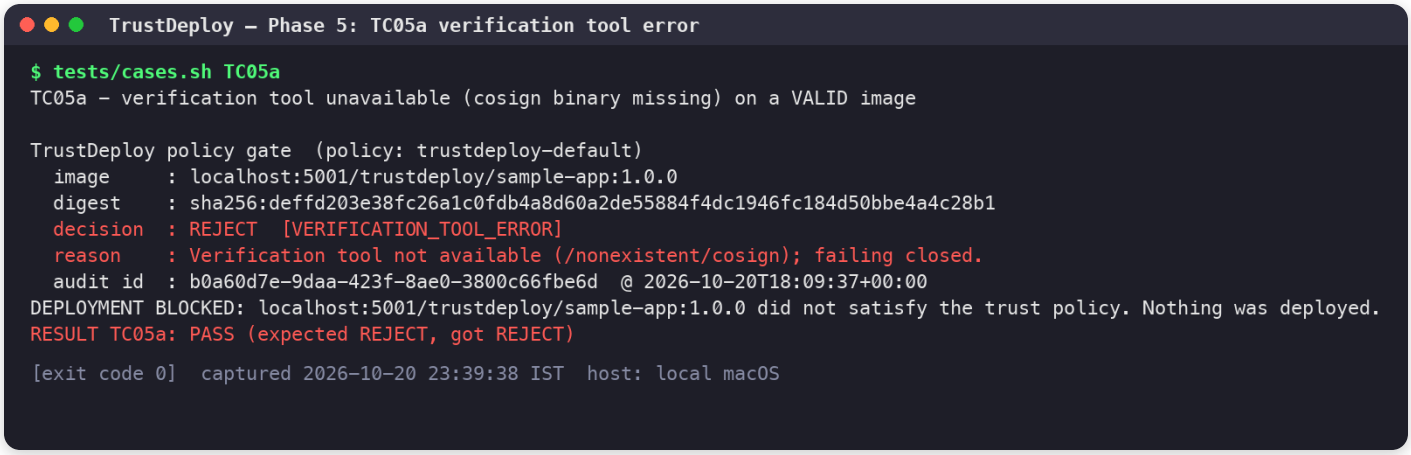
10. TC04b — Trusted tag overwritten with a tampered build

The attacker re-points :1.0.0 to a tampered build. The gate resolves the tag to the new digest, which carries no trusted signature, and rejects it. The tag is then restored.



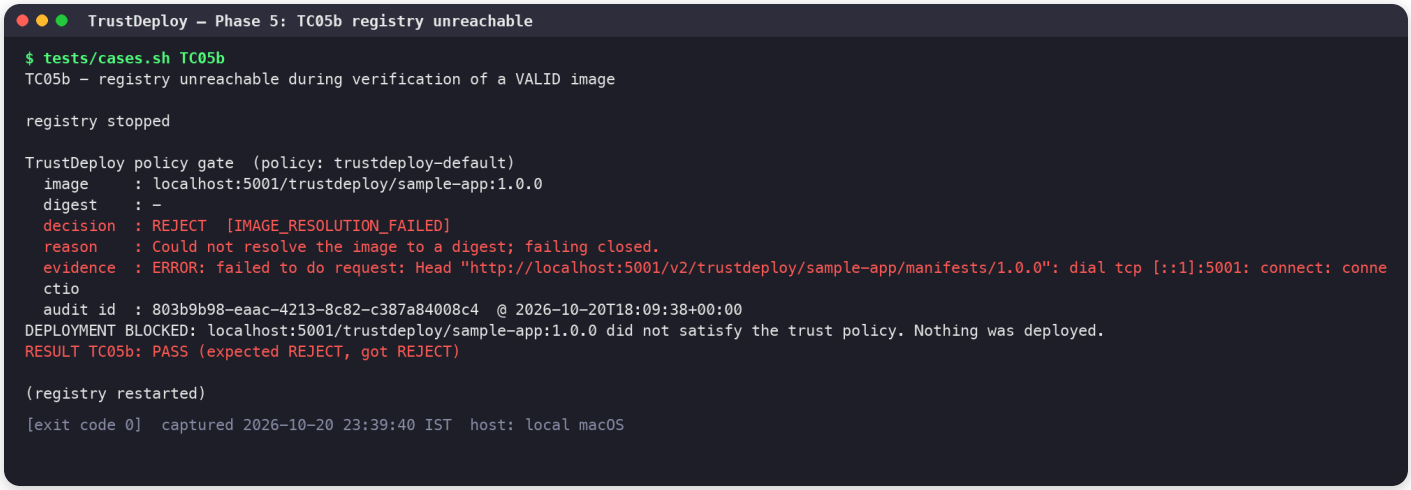
11. TC05a — Verification tool unavailable

Even for a validly signed image, a missing cosign binary results in VERIFICATION_TOOL_ERROR. The gate fails closed and deployment is blocked. Result: TC05 passed.



12. TC05b — Registry unreachable

With the registry stopped, the digest cannot be resolved, so the gate returns IMAGE_RESOLUTION_FAILED and blocks deployment.



13. TC05c — Registry outside the allow-list

docker.io is not in policy.json's allowed_registries, so the gate rejects the image before contacting the registry.

```
TrustDeploy — Phase 5: TC05c registry not allowed

$ tests/cases.sh TC05c
TC05c - image from a registry outside the policy allow-list

TrustDeploy policy gate  (policy: trustdeploy-default)
  image     : docker.io/library/nginx:latest
  digest    : -
  decision  : REJECT  [REGISTRY_NOT_ALLOWED]
  reason    : Registry 'docker.io' is not in the policy allow-list.
  audit id  : ae417f14-9f1d-44c4-8098-4c0fbffe508a  @ 2026-10-20T18:09:40+00:00
RESULT TC05c: PASS (expected REJECT, got REJECT)

[exit code 0]  captured 2026-10-20 23:39:40 IST  host: local macOS
```

14. TC06 — Multiple images, separate audit records

Four images are checked in one batch, and four distinct audit entries are written, each with its own event ID, digest, decision and reason. Result: TC06 passed.

```
TrustDeploy — Phase 5: TC06 batch, separate audit records

$ tests/cases.sh TC06
TC06 - several images checked in one batch; each decision recorded separately

  image     : localhost:5001/trustdeploy/sample-app:1.0.0
  decision  : ACCEPT  [SIGNATURE_VERIFIED]

  image     : localhost:5001/trustdeploy/sample-app:1.1.0-unsigned
  decision  : REJECT  [NO_SIGNATURE]
  reason    : The image has no Cosign signature in the registry.

  image     : localhost:5001/trustdeploy/sample-app:1.0.1-attacker
  decision  : REJECT  [UNTRUSTED_SIGNER]

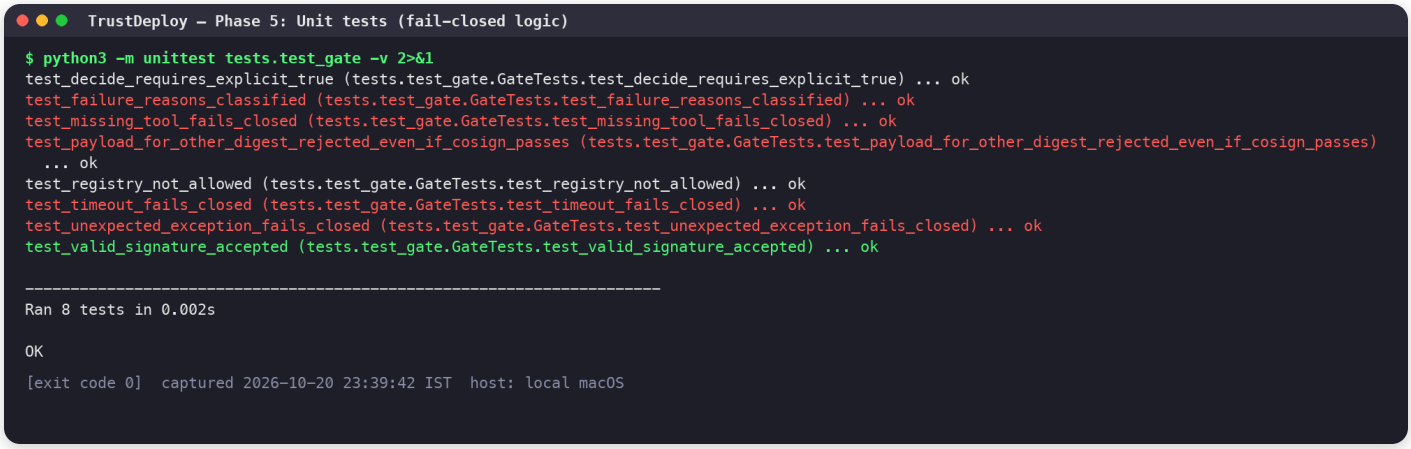
  image     : localhost:5001/trustdeploy/sample-app:1.0.2-forged
  decision  : REJECT  [SIGNATURE_IMAGE_MISMATCH]
  reason    : A signature exists, but it was issued for a different image digest.

audit entries added: 4 (expected 4)
  b1aac8a8  2026-10-20T18:09:40+00:00  TC06   sample-app:1.0.0             ACCEPT  SIGNATURE_VERIFIED
  da519f3c  2026-10-20T18:09:41+00:00  TC06   sample-app:1.1.0-unsigned    REJECT  NO_SIGNATURE
  b1016abc  2026-10-20T18:09:41+00:00  TC06   sample-app:1.0.1-attacker    REJECT  UNTRUSTED_SIGNER
  53ee9525  2026-10-20T18:09:41+00:00  TC06   sample-app:1.0.2-forged      REJECT  SIGNATURE_IMAGE_MISMATCH
RESULT TC06: PASS

[exit code 0]  captured 2026-10-20 23:39:42 IST  host: local macOS
```

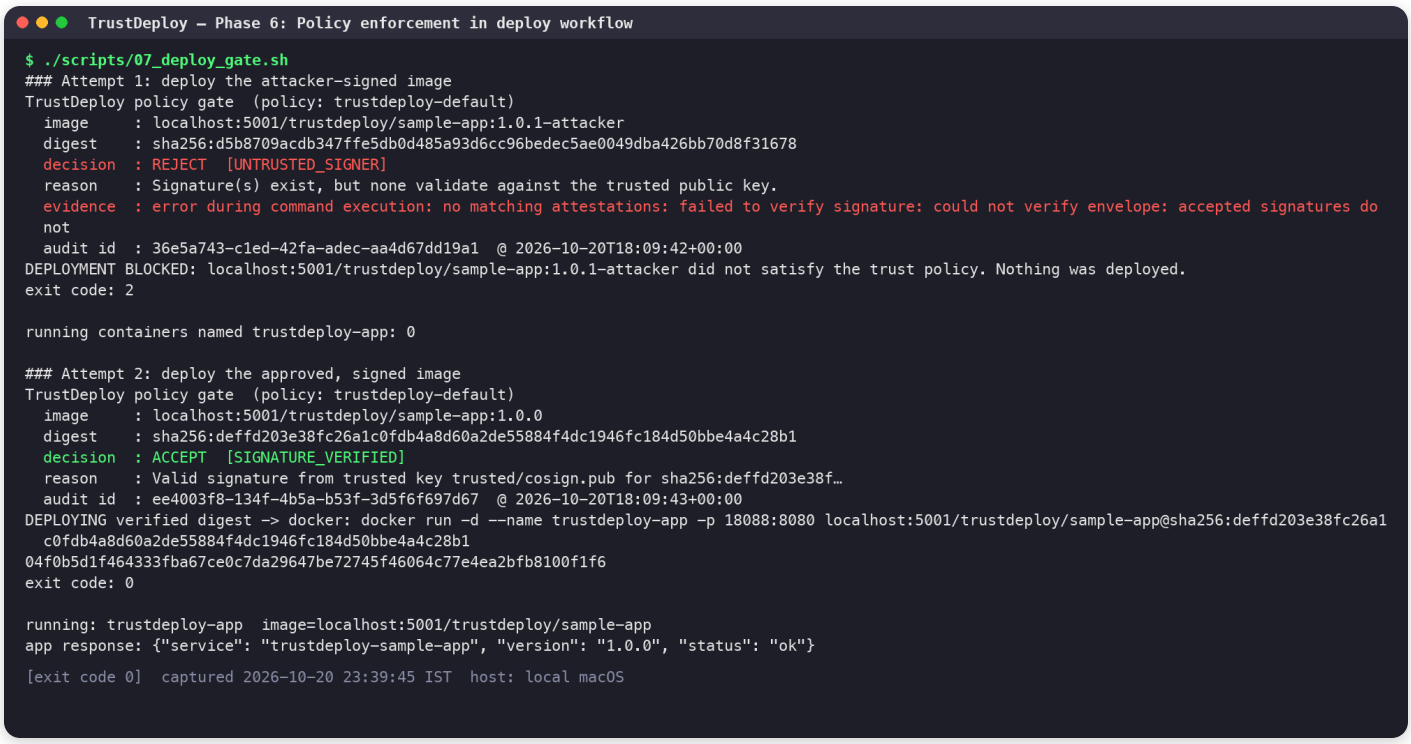
15. Unit tests — fail-closed logic

Eight unit tests, using a mocked cosign, cover acceptance, payload-digest mismatch, error classification, missing tool, timeout, unexpected exceptions, the registry allow-list, and the rule that the decision requires verified to be exactly True. All 8 passed.



16. Phase 6 — Policy enforcement in the deploy workflow

trustdeploy deploy blocks the attacker-signed image (exit code 2, nothing started). It deploys the trusted image pinned to the verified digest, and the running container answers on port 18088.



17. Phase 6 (K8s) — kind cluster + Sigstore Policy Controller

A local kind cluster runs policy-controller 0.13.1 (Helm chart 0.10.8). The ClusterImagePolicy trustdeploy-signed-images requires the TrustDeploy release key for td-registry.local:5000/**, and the trustdeploy namespace is opted in.

TrustDeploy — Phase 6: kind + Sigstore Policy Controller

```
$ kubectl get nodes -o wide | cut -c1-110; echo; helm list -n cosign-system; echo; kubectl -n cosign-system get pods; echo; cat k8s/cluster-image-poli
  cy.yaml
NAME                       STATUS   ROLES           AGE     VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE
trustdeploy-control-plane   Ready    control-plane   4m53s   v1.37.0   172.20.0.2    <none>        Debian GNU/

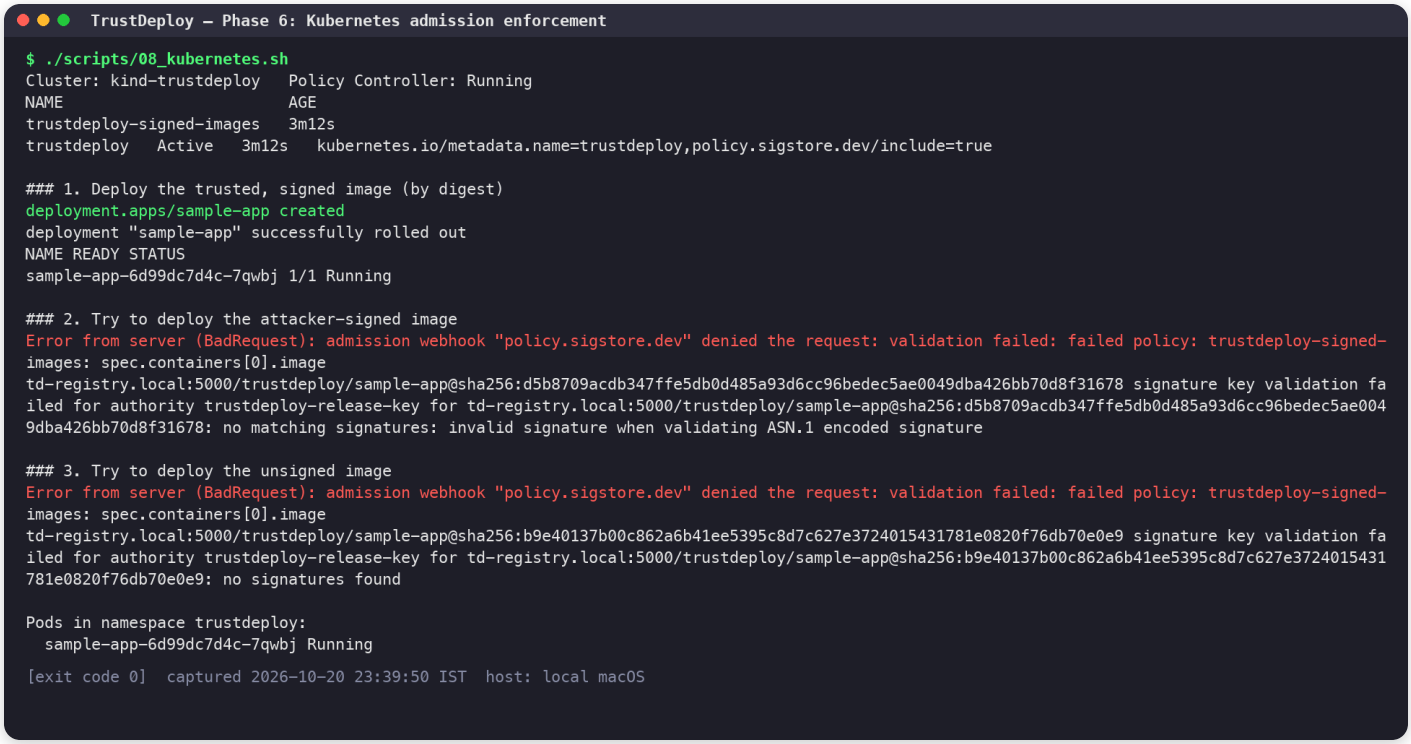
NAME               NAMESPACE       REVISION    UPDATED                                 STATUS      CHART                       APP VERSION
policy-controller   cosign-system   1           2026-10-20 23:35:31.600543 +0530 IST    deployed    policy-controller-0.10.8    0.13.1

NAME                                      READY   STATUS    RESTARTS   AGE
policy-controller-webhook-85d98dd944-q7tbz   1/1     Running   0          4m15s

# Sigstore Policy Controller: only admit images from the TrustDeploy registry that are
# signed by the TrustDeploy release key. Applies to namespaces labelled policy.sigstore.dev/include=true.
apiVersion: policy.sigstore.dev/v1beta1
kind: ClusterImagePolicy
metadata:
  name: trustdeploy-signed-images
spec:
  images:
    - glob: "td-registry.local:5000/**"
  authorities:
    - name: trustdeploy-release-key
      key:
        hashAlgorithm: sha256
        data: |
          -----BEGIN PUBLIC KEY-----
          MFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAEY2b6FmBx0DgxS6U62bBEmCVBe5CK
          AV4VHFUgHPELCYOUwm5WMSa4Y9tspN9Mj7+8kKT+Adohn6AeA8R/AHH3BA==
          -----END PUBLIC KEY-----
[exit code 0]  captured 2026-10-20 23:39:46 IST  host: local macOS
```

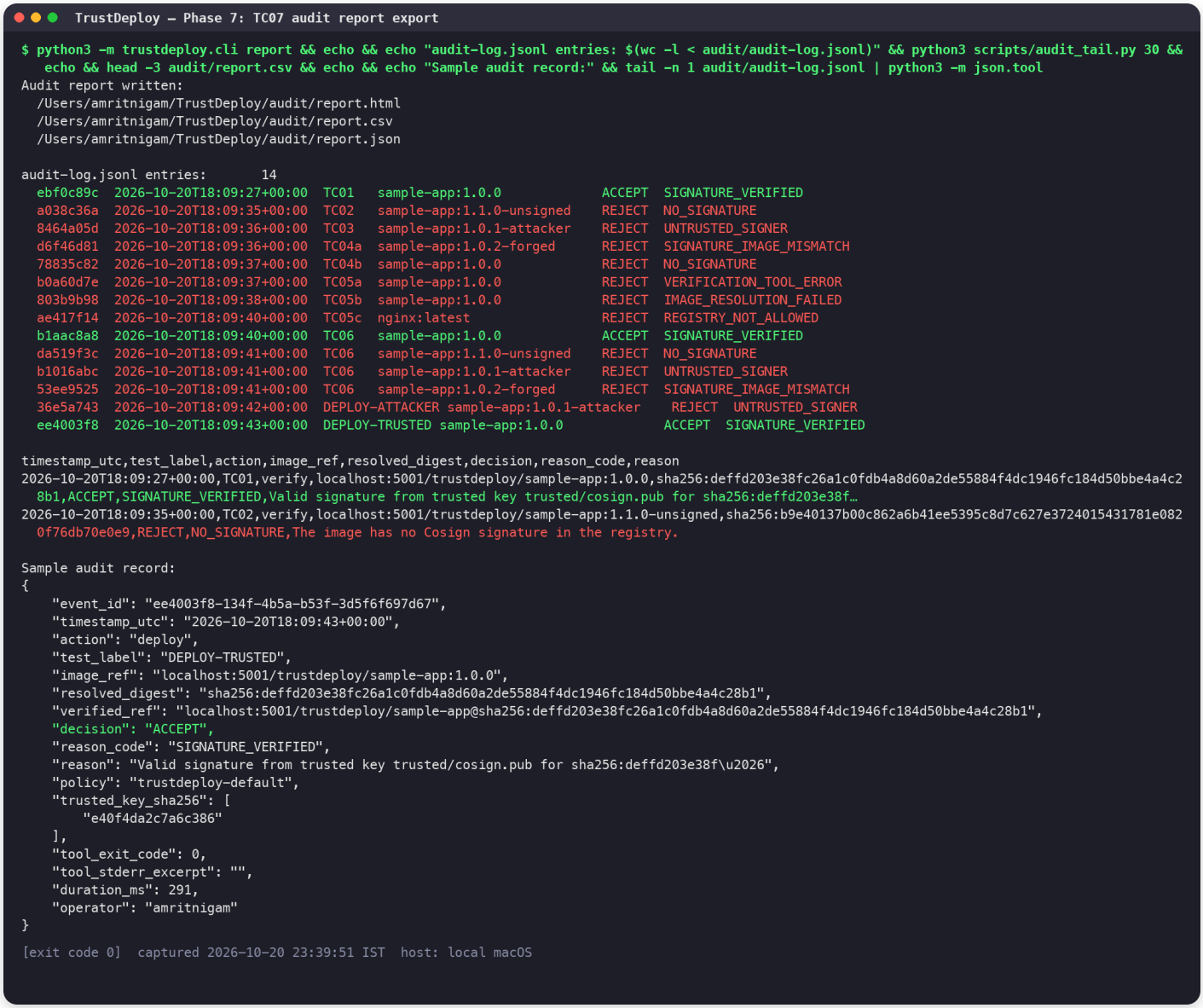
18. Phase 6 (K8s) — admission-time enforcement

The signed image is admitted and runs. The admission webhook denies the attacker-signed image ('invalid signature') and the unsigned image ('no signatures found'), so no pods are created for them.



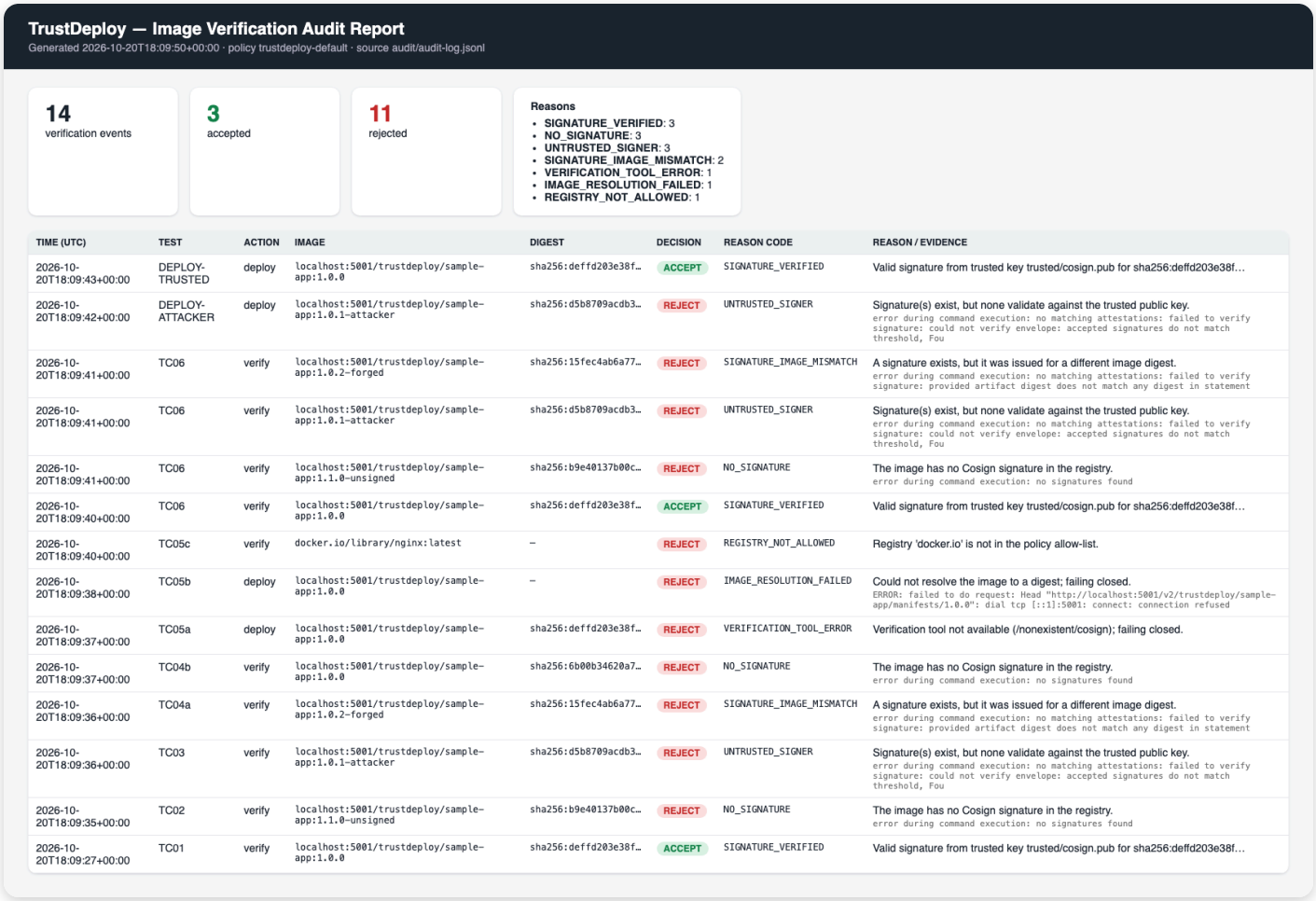
19. Phase 7 — TC07 audit report export

The audit log (JSONL) is exported to audit/report.html, report.csv and report.json. Each record holds the timestamp, image, digest, decision, reason code, policy, trusted-key fingerprint and a tool-evidence excerpt, and never any key material. Result: TC07 passed.



20. Phase 7 — Audit dashboard

The HTML dashboard summarises 14 verification events: 3 accepted and 11 rejected, broken down by reason, with per-event evidence.



TrustDeploy — Image Verification Audit Report

Generated 2026-10-20T18:09:50+00:00 · policy trustdeploy-default · source audit/audit-log.jsonl

14 verification events

3 accepted

11 rejected

Reasons

- **SIGNATURE_VERIFIED**: 3
- **NO_SIGNATURE**: 3
- **UNTRUSTED_SIGNER**: 3
- **SIGNATURE_IMAGE_MISMATCH**: 2
- **VERIFICATION_TOOL_ERROR**: 1
- **IMAGE_RESOLUTION_FAILED**: 1
- **REGISTRY_NOT_ALLOWED**: 1

TIME (UTC)	TEST	ACTION	IMAGE	DIGEST	DECISION	REASON CODE	REASON / EVIDENCE
2026-10-20T18:09:43+00:00	DEPLOY-TRUSTED	deploy	localhost:5001/trustdeploy/sample-app:1.0.0	sha256:deffd203e38f…	ACCEPT	SIGNATURE_VERIFIED	Valid signature from trusted key trusted/cosign.pub for sha256:deffd203e38f…
2026-10-20T18:09:42+00:00	DEPLOY-ATTACKER	deploy	localhost:5001/trustdeploy/sample-app:1.0.1-attacker	sha256:d5b8709acdb3…	REJECT	UNTRUSTED_SIGNER	Signature(s) exist, but none validate against the trusted public key. error during command execution: no matching attestations: failed to verify signature: could not verify envelope: accepted signatures do not match threshold, Fou
2026-10-20T18:09:41+00:00	TC06	verify	localhost:5001/trustdeploy/sample-app:1.0.2-forged	sha256:15fec4ab6a77…	REJECT	SIGNATURE_IMAGE_MISMATCH	A signature exists, but it was issued for a different image digest. error during command execution: no matching attestations: failed to verify signature: provided artifact digest does not match any digest in statement
2026-10-20T18:09:41+00:00	TC06	verify	localhost:5001/trustdeploy/sample-app:1.0.1-attacker	sha256:d5b8709acdb3…	REJECT	UNTRUSTED_SIGNER	Signature(s) exist, but none validate against the trusted public key. error during command execution: no matching attestations: failed to verify signature: could not verify envelope: accepted signatures do not match threshold, Fou
2026-10-20T18:09:41+00:00	TC06	verify	localhost:5001/trustdeploy/sample-app:1.1.0-unsigned	sha256:b9e40137b00c…	REJECT	NO_SIGNATURE	The image has no Cosign signature in the registry. error during command execution: no signatures found
2026-10-20T18:09:40+00:00	TC06	verify	localhost:5001/trustdeploy/sample-app:1.0.0	sha256:deffd203e38f…	ACCEPT	SIGNATURE_VERIFIED	Valid signature from trusted key trusted/cosign.pub for sha256:deffd203e38f…
2026-10-20T18:09:40+00:00	TC05c	verify	docker.io/library/nginx:latest	–	REJECT	REGISTRY_NOT_ALLOWED	Registry 'docker.io' is not in the policy allow-list.
2026-10-20T18:09:38+00:00	TC05b	deploy	localhost:5001/trustdeploy/sample-app:1.0.0	–	REJECT	IMAGE_RESOLUTION_FAILED	Could not resolve the image to a digest; failing closed. ERROR: failed to do request: Head "http://localhost:5001/v2/trustdeploy/sample-app/manifests/1.0.0": dial tcp [::1]:5001: connect: connection refused
2026-10-20T18:09:37+00:00	TC05a	deploy	localhost:5001/trustdeploy/sample-app:1.0.0	sha256:deffd203e38f…	REJECT	VERIFICATION_TOOL_ERROR	Verification tool not available (/nonexistent/cosign); failing closed.
2026-10-20T18:09:37+00:00	TC04b	verify	localhost:5001/trustdeploy/sample-app:1.0.0	sha256:6b00b34620a7…	REJECT	NO_SIGNATURE	The image has no Cosign signature in the registry. error during command execution: no signatures found
2026-10-20T18:09:36+00:00	TC04a	verify	localhost:5001/trustdeploy/sample-app:1.0.2-forged	sha256:15fec4ab6a77…	REJECT	SIGNATURE_IMAGE_MISMATCH	A signature exists, but it was issued for a different image digest. error during command execution: no matching attestations: failed to verify signature: provided artifact digest does not match any digest in statement
2026-10-20T18:09:36+00:00	TC03	verify	localhost:5001/trustdeploy/sample-app:1.0.1-attacker	sha256:d5b8709acdb3…	REJECT	UNTRUSTED_SIGNER	Signature(s) exist, but none validate against the trusted public key. error during command execution: no matching attestations: failed to verify signature: could not verify envelope: accepted signatures do not match threshold, Fou
2026-10-20T18:09:35+00:00	TC02	verify	localhost:5001/trustdeploy/sample-app:1.1.0-unsigned	sha256:b9e40137b00c…	REJECT	NO_SIGNATURE	The image has no Cosign signature in the registry. error during command execution: no signatures found
2026-10-20T18:09:27+00:00	TC01	verify	localhost:5001/trustdeploy/sample-app:1.0.0	sha256:deffd203e38f…	ACCEPT	SIGNATURE_VERIFIED	Valid signature from trusted key trusted/cosign.pub for sha256:deffd203e38f…